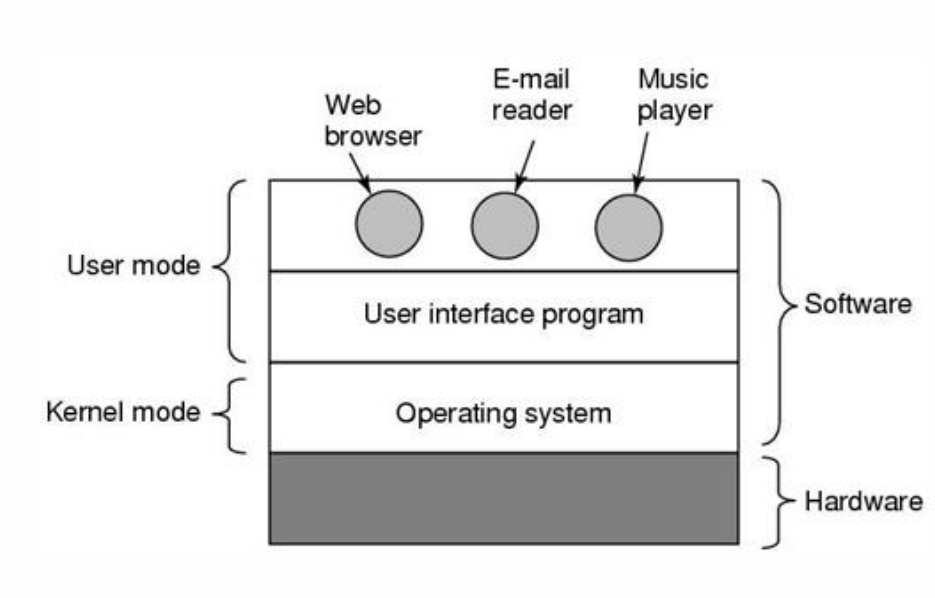
操作系统

概述

什么是OS——hide complexity of h/w

- An OS is the largest and most complicated software（大）
- An OS creates a computer that is convenient and easy to use from the underlying hardware(方便使用）
- OS is between the applications and the hardware（位于哪里）



Summary - OS is about Illusions

- Several programs are running simultaneously
- My program has all the memory
- Disk is organised in files
- Disk access is quick 预读（
- Storage devices work the same 设备驱动
- … etc.

OS creates an ‘ideal’ computer from a real one

User mode vs Kernel mode——prevent interfere

- Kernel: “The one program running at all times“
- Kernel mode：
  - The kernel is the core of the OS
  - Kernel has complete control over everything
- User mode：
  - user programs and apps operate in user mode
- Dual mode operation prevents user data from interfering with kernel data

Variety of Operating Systems

- **Mainframes** 大型机 (Mainframes)
  - IBM z/OS, also Linux, Solaris
- **Servers**
  - Linux (2/3) and Windows (1/3)
- **Supercomputers**
- **PCs (desktop, laptop)**
- **Phones and PDAs** PDA：掌上电脑，手机前身
- Real-time embedded systems
- **Wearable devices**, InternetOfThings(IoT)
  - AndroidWear, Tizen, etc.

关系

Linux / Unix main example on this module

核心概念

- Process：
  - 目的：多程序，多用户，少CPU
  - Illusion of multiple processes（进程切换）
  - Concurrency： play music while editing your code（逻辑层面）
  - parallelism：distribute a single task across multiple processors in order to go faster（物理层面）
- Memory：
  - 目的：多进程共享内存
  - Illusion of your program can use all the memory（虚拟内存）
  - Virtual memory： Memory as it appears to the program
  - What is really happening underneath
- 文件
  - Illusion: disk is organised into files（文件系统）
  - 层次结构
  - 权限
  - 属性（inode属性+文件名）

IO中断

- 硬件层面中断
  - CPU把输入指令给到设备控制器
  - 设备控制器读取完成数据，通知中断控制器
  - 中断控制器通知CPU，并传向量号，告诉CPU应该如何处理这个中断
- 软件层面中断
  - 设备(CPU) 告诉设备驱动(找到了中间的**中断向量表 (IDT))**
  - 驱动返回唤醒进程（处理中断）
  - 若是Programmed I/O,还需要去搬东西去

OS的接口

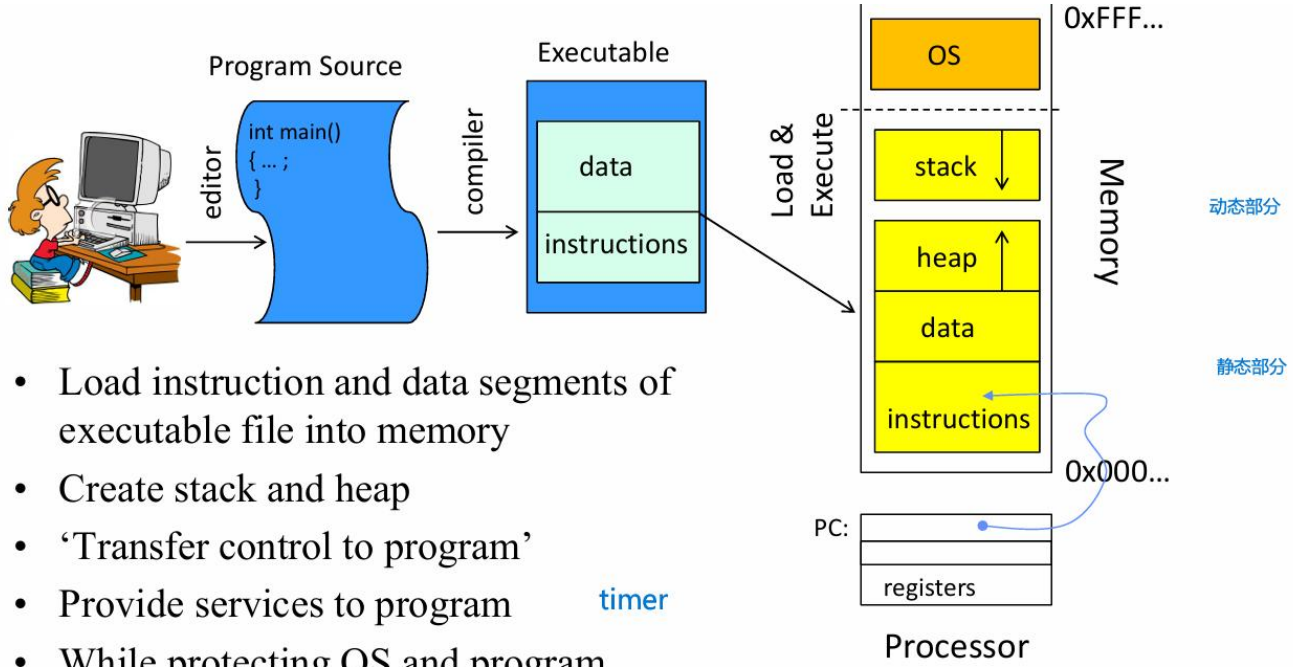
- Command Line
- System Calls: Programming interface to OS
  - POSIX
  - system call is one way to switch from user mode to kernel mode
  - you use a function that evokes a system call
- 完整顺序：User->standard utility programs (complier)->standard library (open) -> OS -> hardware

- ls *list the directory*
- mkdir *make a new directory*
- cd *change current directory*
- pwd *show current directory*
- rm *delete a file*
- rmdir *delete a directory* 必须空
- link *create a symbolic link*
- cp *copy a file*
- mv *move / rename a file*
- cat *look at a file*
- head *look at the start of a file*

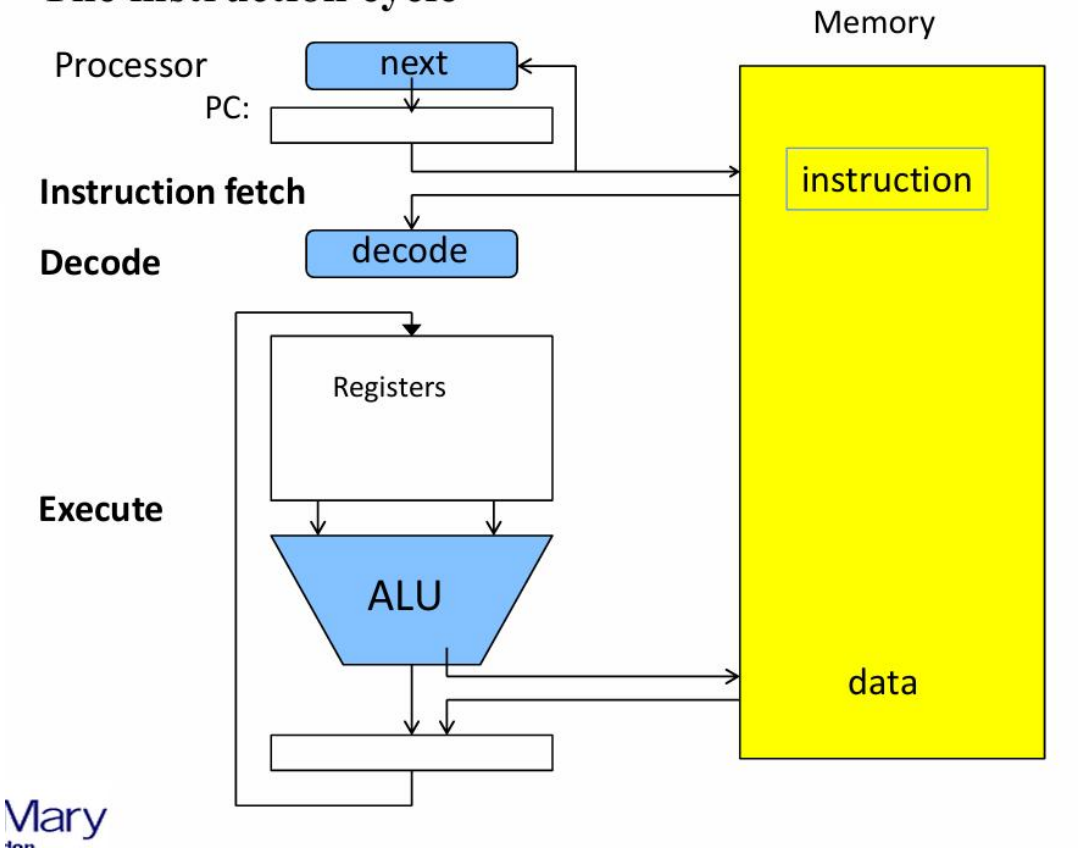
Process

Process——all about interleave

- A process is an instance of a running program
  - current state（pc, sp, other registers) / system resources (memory, files)
- 如何运行一个程序:
  - 1.加载指令和数据到内存
  - 2.创建堆栈作为动态存储
  - 3.PC先指向第一条指令，进而不断向前，“Transfer control to Program”，并解码指令（fetch，decode）
  - 4.CPU读写数据到寄存器，通过算数单元计算（execute）
  - 5.OS给程序提供服务和保护



- Load instruction and data segments of executable file into memory
- Create stack and heap
- ‘Transfer control to program’
- Provide services to program timer
- While protecting OS and program



Process：共享资源

- 进程与OS：OS需要记录Process的信息（PCB和全局信息，属于kernel memory）
  - Resume process
  - scheduling
  - Clean resources
- 进程与内存：空间上共享内存，内存需要存储进程的数据和元数据
  - Program code（instruction）
  - Data
  - 上下文process context（PCB）
  - PCB中包含base Limit等实现地址转化的功能
- 进程与CPU：时间上共享CPU
  - 通过时间切片实现多进程
  - 需要一个结构保存进程对应的寄存器信息（PC，SP）-PCB
- 进程与寄存器
  - PC，SP，内存页表等等，切换时保存在PCB中
- 进程与文件
  - fp 和 指向进程级打开文件表的指针

进程状态

- Two-State Process Model：Running/Non-running
- Five-State Process Model：加入New，Exit，把Not Running分为Ready和Blocked (may be waiting for I/O等event)
- Multiple Queues
  - Ready Queue
  - Event Queue 一个设备一个队列，方便唤醒，但具体先运行哪个是disk schedule问题
- Suspending a process（七个状态）
  - 加入Blocked/Suspend和Ready/Suspend
  - Suspend a process: Swap it out of memory（硬盘）

Process Creation & Termination

New Process Creation
Login
New batch job
Created by OS to provide service
Spawned by existing process

Termination
Normal Completion
Memory unavailable
Protection error
Operator or OS Intervention
Arithmetic error (e.g. divide by 0)
Privileged instruction
etc.

Process Control Block

- In Linux the kernel stores the list of processes in a double linked list
- Kernel represents each process as a process control block (PCB)
- 信息包括PC和stack pointer, values of register (Enough information to resume an process)
- 还包括调度信息: Priority, Execution time, State, id

Protection

- The OS must protect itself from user processes
  - Reliability: compromising the operating system usually causes a crash
  - Security: limit the scope of what processes can do
  - Privacy: limit each process to the data it is permitted to access
  - Fairness: each should be limited to its appropriate share of resources (CPU time, memory, I/O, etc.)
- It must protect processes from one another
- Mechanism: address translation; Dual mode operation; System call

User vs. Kernel Mode（Mode switch）

- 模式切换标志-a bit of state (system mode bit)
- 模式切换-也需要保存一部分上下文
  - User⟶ Kernel transition sets system mode AND saves the user PC
  - Kernel ⟶ User transition clears system mode AND restores appropriate user PC and other state data
- 最需要小心，重要的-user到kernel
- 三种模式切换的方式:
  - System call-Process requests a system service
    - 通过函数调用的方式（比如标准库中的read）
  - Interrupt-External event triggers
  - Exception-Internal event triggers

Process Switch

- Switch Process 需要Mode switch，反过来不一定
- 切换方式-同模式切换（3个）
- 过程-保存上一个进程的状态，更新PCB，移到其他队列中
  -选择一个新进程，更新PCB，更新内存管理，恢复进程上下文
- 需要很大的overhead
  - Lots of data to store/load
  - Scheduling
  - Cache reloading

Mechanism	Cause	Use
Interrupt	External to the execution of the current instruction	Reaction to an asynchronous external event
Trap	Associated with the execution of the current instruction	Handling of an error or an exception condition
Kernel call	Explicit request	Call to an operating system function

Process2

Scheduling: Aims

- Throughput: Maximise the amount of processes completed in a given time frame
- (Prevent)Starvation: a process that does not get executed for too long
- Responsive time
- Fairness
- Utilization: use the resources efficiently (e.g. avoiding idle time for the CPU)

Process Scheduling

- aims：Share time fairly among processes
  - 平衡batch process和Interactive process
  - Batch process : Computationally/CPU intensive，Throughput优先，long CPU burst
  - Interactive process：I/O intensive，Response time matters most，short CPU burst
  - CPU burst的分布：大量进程是short CPU burst，即Interactive process
  - 现代调度器利用分布，give precedence to short processes，computationally heavy will probably not ‘notice
- Non-preemptive vs. Preemptive scheduling
  - 非抢占式-Once a process is in the running state it continues until completed or give up the CPU voluntarily
  - 抢占式进程-The currently running process may be interrupted
    - when a new process arrives
    - on an (I/O) interrupt
    - periodically based on a timer

Process Scheduling

- 类别
  - FCFS：Non-preemptive，进程只有自愿或需要输入输出才会放弃占用CPU，先请求CPU的先访问
    - 对CPU密集型友好，对短进程不友好
    - 对进程burst time差不多的比较友好，简单实现（单链表队列）
    - Simple to implement (linked list queue)
  - SJF：短进程优先，需要考虑不同进程在不同时间达到的问题
  - RR：用Time Slicing，better for shorter jobs and jobs are of varying lengths
    - More fair than FCFS
    - q太大-poor responsiveness
    - Q太小-overhead of context switch
- 进程调度术语
  - Arrival Time，Burst Time，（平均）Completion Time
  - Turn Around Time 周转时间 CT-AT
  - （平均）WT waiting time：所有等待的时间，包括相应和中间等待（TAT-BT）
  - RT Response time：CPU第一次使用-AT

Multi-level Queue (Scheduling with Priorities)

- Multi-level Priority Queuing
  - starts at highest priority queue
  - using some scheduling policy (e.g. RR) in one priority queue
  - Lower-priority processes may suffer starvation -> priority of a process can change with its age (根据时间改变优先级)
- Multi-Level Feedback Queues（根据结果改变优先级）
  - A process starts at the top level of priority'
  - On pre-emption: go down a level; On I/O block: go up a level
  - possible for starvation to occur -> vary the preemption times according to the queue

Process Control

- fork(): system call to create a copy of the current process, and start it running
  - 返回0-子进程
  - 大于0-父进程，返回的是子进程pid
  - 小于0-错误，还在原来进程运行
  - Python fork-没创建新的进程，报异常
  - Copy On Write. The duplication occurs only when there is attempt to write on the shared copy
- os.execv: system calls to change the program being run by the current process
- os.waitpid(): system call to wait for a process to finish
  - 第一个参数0是等待进程组的子进程，-1就是必须是自己的子进程
  - 返回子进程退出状态
- signal.signal(): system call to send a notification to another process

CFS（Completely Fair Scheduler）

- Niceness【-20，19】，0是默认，越高越谦让（越不优先）
- Time Slice：一个周期内一个进程希望连续运行的时间，根据权重进行分配，权重是1/1.25^niceness
- Sched_min_granularity-最小的time before preemption
- CFS调度-根据vRuntime调度，最小的优先，vRuntime是算权重的
- 层次调度：用户-进程-线程
- 用红黑树存储vruntime-直接访问最左边

CFS (new)

- CFS: always pick least served task so-far (smallest value of vruntime)
- Time slice: share of scheduler period proportional to task weights
- Hierarchical Scheduler (User->Process->Thread)
- Log-time Queue implemented with Red-Black Tree

文件系统

文件系统

- 文件系统是什么？ layer of OS that transforms block interface of disks into Files, Directories, manage all the files on physical drive.
- 文件系统的目标:
  - File management: storage
  - Space management: allocation
  - Access control: permission
  - Data Integrity: no error
- 一些term
  - File: A collection of data stored together
  - Directory: A container for organizing files
  - Metadata: Information about the file: 比如permission and location
  - Block/sector: The smallest unit of storage on a disk
  - Inode: A data structure that stores metadata

文件系统

- 文件系统的分类：FAT（老的）； NTFS（Windows）； EXT4（Linux）； HFS+（Mac）； exFAT（外接设备）
- System View： File System-> Block
- User View: User->file->folder
- 如何访问文件： file path->(Diretory Structure)->File number(inode号）-> （inode）->File Index Structure ->Data blocks

文件

- 文件是什么？ an abstraction for non-volatile storage
  - A ‘named’ permanent storage
  - A logical unit of information created by some process
- 文件包含什么？
  - Data
  - Metadata (size, owner, time, access rights & location)
- 要求
  - Variable size
  - Multiple concurrent with protection
  - Manage free disk blocks (allocation)
  - Being able to find the file
- File type: OS must recognise its own executable files (magic number)

目录（Directories）

- Filename stored in a directory , also mapping to pointer to inodes (system file ID/Inode number)
- It is also a file
- Path Names
  - Absolute path name: The whole path from the root directory to the file or directory
  - Relative path name: In relation to the current directory you are in

打开文件表

- System-wide open file table: information for every currently open file in the system (inode information)
- Per-process open file table: containing a pointer to the system open file table as well as some other information
- Open file
  - reads in the inode information, stores it in system-wide open file table.
  - an entry is added to the per-process open file table
  - index into the per-process table is returned (file descriptor)
- Close file
  - Per-process table entry is freed
  - Memory cache data written out to disk

Python中的文件

File mode	Description
w	Create a file for writing
x	Open a file for exclusive creation
a	Open the file in append mode
b	Create a binary file
t	Create and open a file in text mode

Python中的文件

读，删，查看存在

- os: The os module provides a way to interact with the operating system. It includes functions for file operations like renaming, deleting, checking existence, and more.
- os.path: This module is part of the os package and provides functions for common pathname manipulations. It's particularly useful for checking file paths, splitting paths, joining paths, etc.

路径

- shutil: The shutil module offers a higher-level interface for file operations than os. It provides functions for copying files, moving files, archiving, etc.

复制，移动压缩

- glob: The glob module is used to search for files that match a specified pattern. It's helpful for finding files with specific extensions or patterns within directories.

通配符

extensions 扩展名

- pathlib: Introduced in Python 3.4, pathlib provides an object-oriented approach to file system paths. It's more intuitive and easier to use than the traditional string-based methods.

java的FILE

PHP中的文件

- opendir — Open directory handle
- closedir — Close directory handle
- readdir — Read entry from directory handle
- rewinddir — Rewind directory handle

句柄，有点像二进制文件的指针

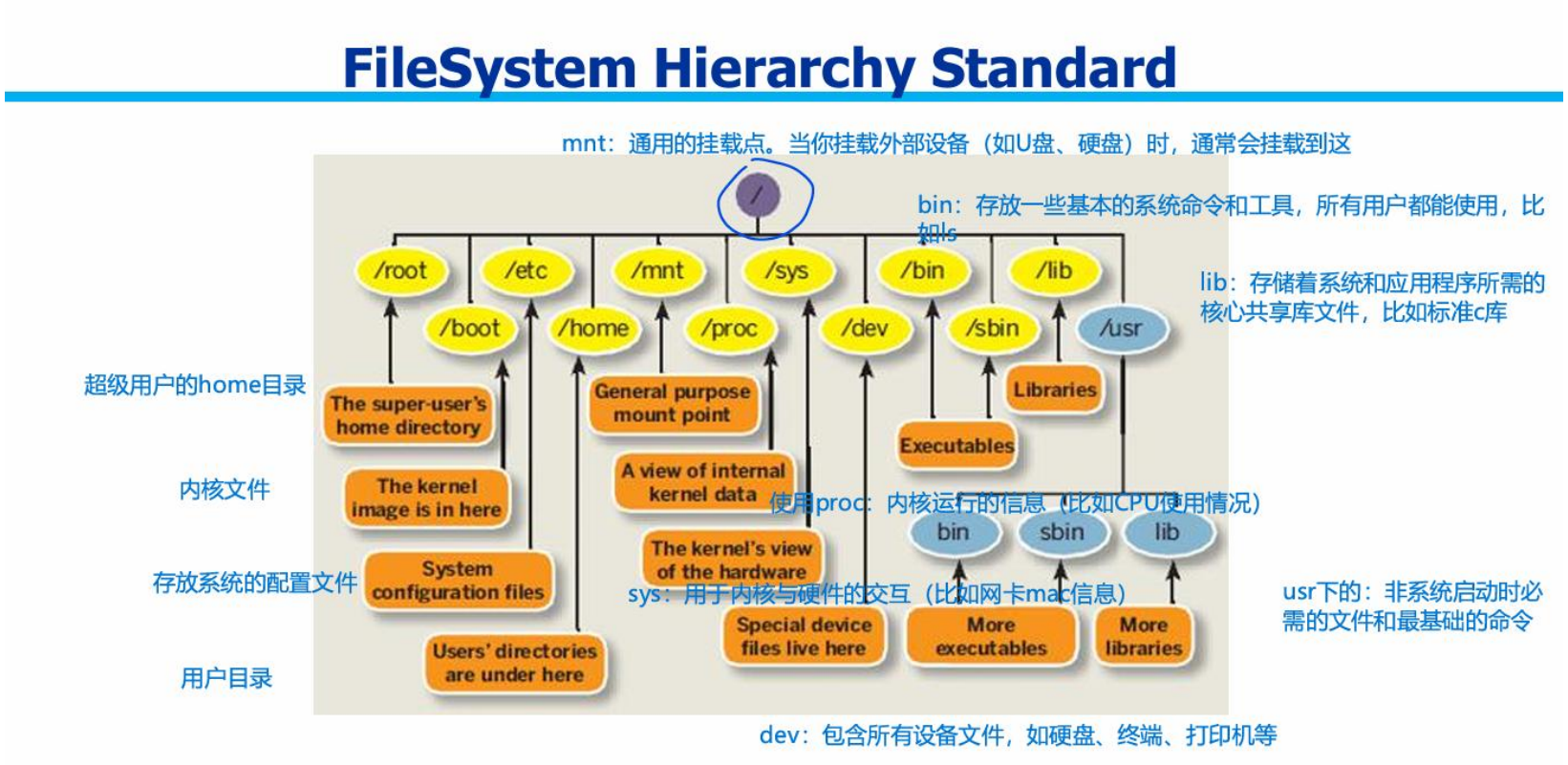
重置

fseek — Seeks on a file pointer

ftell — Returns the current position of the file read/write pointer

文件共享和文件系统层级

- Locks: Management of simultaneous access
- Permissions： chmod 740 text.txt



Link

- Linux command ‘link’ (‘ln’) allows us to have two names for same data
- Hard link- Linked files share the same inode number; Soft link- File contains (full path) name of the original file

- **Key Differences Between Hard and Soft Links**
- **Inode**: Hard links share the same inode as the original file, while soft links have a unique inode.
- **File System**: Hard links are restricted to the same filesystem, whereas soft links can cross filesystems.
- **Behavior on Deletion**: Deleting the original file breaks a soft link but does not affect hard links

- **Benefits**
- **Hard Links**: Useful for creating multiple references to critical files within the same filesystem, ensuring data persistence even if one link is deleted.
- **Soft Links**: Ideal for creating shortcuts, linking files across filesystems, or managing versioned files (e.g., libraries).

file allocation(which disk blocks go with which file)

- Contiguous allocation
  - Sequence of blocks allocated at file creation
  - Problems: Extenal fragmentation; Dynamic ‘growth’ of files is not fully supported
  - Good things: Easy implementation（start address + Length); Good performance(1 seek)
- Linked Allocation（FAT）:三层映射
  - FAT is linked list 1-1 with blocks
  - File Number is index of root of block list for the file
  - Follow list to get block
  - FAT free list
  - simple to implement but slow to find blocks (随机访问效果很差)
- Indexed Allocation（inode）
  - 几重指针（小文件，直接访问；大文件，层层查找）
  - 一般使用bitmap来存储free space

Inode

- 作用
  - Keeps information about files (metadata)
  - Keeps information about which disk blocks belong to which file
- Inode信息
  - 文件大小(The size of the file)
  - 设备ID (Device ID)
  - 用户，Group ID和访问权限（file mode）
  - 时间戳（修改inode时间，修改内容时间，访问时间）(Timestamps)
  - 硬连接数(A link counter)
  - 指向存储的block指针
- 存放位置
  - Outermost cylinder-> closer to the data itself，performance&reliability都不行
- File type: The file may be of the following types: Regular files, special files, directories, or symbolic links.

Actual File Systems

- ZFS is a combined file system and logical volume manager
- ExFAT designed for USB sticks supports large files speeds up free space management with bitmaps
- Journaling File Systems： put writing operations into a journal before executing them to recover more easily

老windows 老linux

	FAT32	EXT2
Largest File	4GB (due to 32bit field for file size)	2TB (for 4KB block size)
Free Space	File Allocation Table (FAT)	Bitmap for block group
File Allocation	Linked Allocation: Linked List for file's clusters in FAT	Index Allocation: 15 pointers to Data Blocks per inode (12 Direct, 1 Indirect, 1 Double-Indirect and 1 Triple-Indirect)
Hard Links?	NO, each "virtual file" maps to exactly one "physical file"	YES, multiple "virtual files" can map to same "physical file" (inode)
Symbolic Links?	PARTIALLY, through special files in OS (shortcuts).	YES.

磁盘和I/O

I/O

- 遇到的困难(challenging)
  - Time it takes to seek, rotate, transfer times
  - 文件不连续存储(no guarantee our files will be stored in contiguous locations)
  - Device rates vary
- BUS: A set of wires (H/W) for communication among devices plus protocols for data transfer operations

Device Controller

- Tasks：
  - Read/write operations
  - Validation/error correction
  - Communicates with the CPU
- Memory-Mapped I/O
  - Some locations in main memory are reserved to store I/O device ‘s buffers and registers
  - reading from and writing to special locations in (main) memory
- OS和I/O交互
  - The OS detect status register
  - The OS writes into the data register and sets the command register
  - The controller reads the command and the data register and launches the execution of the command
  - The OS detects when the command has been executed based on the status register
    OS检测设备状态-》写入指令和数据，设置状态为忙-》控制器执行命令-》OS根据返回状态看是否完成

Interrupts vs. Polling

- Interrupts：Device generates an interrupt whenever it needs service
  - 优点： handles unpredictable events well
  - 缺点： high overhead
- Polling：OS periodically checks a device-specific status register
  - 在内核态时顺便查询
  - 优点： low overhead
  - 缺点： may waste many cycles for unpredictable event

Programmed I/O vs Direct Memory Access

- Programmed I/O： Each byte transferred via processor in/out or load/store instructions
  - 优点： Simple hardware, easy to program （软硬件）；
  - 缺点： Consumes processor cycles proportional to data size
- DMA
  - Give controller access to memory bus
  - Data transfer between memory and device without the CPU
  - The CPU is interrupted only at start and end of the transaction
  - （bus) cycle stealing: Force processor to suspend operation temporarily
  - 过程：
    - 指令： device driver (CPU) -> disk controller
    - 数据： disk controller ->DMA controller -> buffer (memory)

I/O and timing

- Synchronous, Blocking – “Wait”
  - Put the requesting process to sleep until the read/write is done
- Asynchronous, non-blocking – “Tell me later”
  - give pointer to buffer for kernel to fill/take data, return immediately to process（之后会notifies the process）
  - （一般搭配多线程）

Device Drivers

- Device Driver: Device-specific code in the kernel that interacts directly with the hardware
  - Supports a standard, internal interface
  - Same kernel I/O system can interact easily with different device drivers
- Top half: accessed in call path from system calls
  - implements a set of standard, cross-device calls
  - This is the kernel's interface to the device drive
  - Top half will start I/O to device, put process to sleep until finished
- Bottom half: run as interrupt routine
  - Gets input or transfers next block of output
  - May wake sleeping processes

HDD

- Track: ring around the disk surface一圈
- Cylinder: stack of tracks that are the same distance from the spindle 同距离的轨道堆叠
- Sector: segment of track 部分圈，512bytes
  - Sector is the unit of transfer

Disk Scheduling (The device driver)

- Seek times dominate the time taken for read/write operations
- FIFO：The most fair；very long seeks
- SSTF（Shortest Seek Time First）：quite good at reducing seek times; it may lead to starvation
- Scan（elevator algorithm）：It avoids the problem of starvation；not fair; Biased against the area recently passed
- C-Scan：Fairer than SCAN
- Scan的问题
  - Works against locality of reference
  - Anticipatory schedulers：Delay after satisfying a read request to see if a new nearby request

SSDs

- 定义: A device that stores data persistently using integrated circuits without any involvement of moving mechanical parts
- Flash chip structure
  - Chips are organised in banks
  - Banks are divided in blocks (e.g. 256KB) （擦除单元）
  - Blocks are divided into pages (e.g. 4KB)（读写单元）
  - SSDs offer operations on 512 byte “sectors” （物理存储单位）
- Writing
  - Can only write empty pages in a block
  - Writing requires erasing a block
  - The number of times a block can be programmed/erased is limited
    - wear levelling

SSDs vs. HDDs 优势

- random-access I/O, SSDs offer a big advantage.
- more reliable due to the lack of mechanical moving parts (very shock insensitive)
- SSDs also very light weight, low power, silent